

SIMATSENGINEERING

ASSESSMENTTOOL2

COURSE CODE: CSA1403

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

Duration :60 Mins
On Class

QUESTIONS: 25 Total Marks:50

Annexure A

SHORT ANSWER TEST

Code of Conduct:

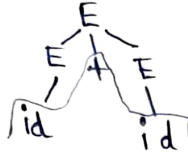
*I Pooja sree.M(192521403)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Pooja sree.M

Signature of the student:

Q1.) For token stream $id + id$, parser validates and check whether the sequence follows grammar rules for an expression.

eg: $E \rightarrow E + E / id$



Hence, the structure is syntactically valid.

2.) $S \rightarrow a^* b$

$S \rightarrow a$

$S \rightarrow aa$

$S \rightarrow aaa$

$S \rightarrow aaab$

$\therefore$ The string belongs to the language.

3.) $A \rightarrow Aa^* b$

$A \rightarrow A\alpha / \beta$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon$

Left Recursion:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon$

4.) $S \rightarrow aA^* b \quad A \rightarrow c / \epsilon$

$\text{First}(S) = \{a, b\}$

5.) $\underline{S} \rightarrow \underline{S} a b$

The grammar is left recursion and so that not suitable for top down parsing.

6.) Left factoring:

a $\rightarrow S \rightarrow \text{if } E \text{ then } S.$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 / \beta_2 / \beta_3$

Ans:

$S \rightarrow \text{if } E \text{ then } S S'$

$S' \rightarrow \text{else } S / \epsilon$

7.) $E \rightarrow E + E$

$E \rightarrow id$

Parse tree for $a + b$.



8.) recursive descent parser:

$S()$

{

if (input == 'a')

{

input++;

A();

}

else if (input == 'b')

{

input++;

}

}

main()

{

S();

if (input == 's')

printf("Parsing Successfull:");

}

9.) $S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

First(S) = {a}

First(A) = {a}

First(B) = {b}

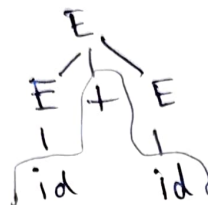
Table:

	a	b
S	$S \rightarrow AB$	
A	$A \rightarrow a$	
B		$B \rightarrow b$

Q1)

Stack	I/P	Action
\$	id + id	shift id
\$ id	id + id	shift Reduce id $\rightarrow$ E
\$ E	+ id	shift +
\$ E +	id	shift id
\$ E + id	id	Reduce id $\rightarrow$ E
\$ E + E	\$	$E \rightarrow E + E$
\$ E	\$	Accept

11) id + id
 Grammar:
 $E \rightarrow E + E$
 $E \rightarrow id$



12) operators: +, *
 $\therefore * > +$ $| + < *$
 * has higher precedence than +.

Precedence Table

operator	+	*
+		<
*	>	

13) id * id
 $\downarrow$
 $E * E$
 $\downarrow$
 E
 $\therefore E \rightarrow E * E$

$\therefore$ The parser first recognize id reduce it to E.

14) LR Parser $\rightarrow$ scans the input from left to right and rightmost derivation is reversed.

$S \rightarrow aA$
 $A \rightarrow b$

Stack	I/P	Action
\$	ab \$	Shift a
\$ a	b \$	shift b
\$ a b	\$	Reduce $b \rightarrow A$
\$ a A	\$	Reduce $aA \rightarrow S$

\$S	\$	Accept
-----	----	--------

15) SLR :

$S \rightarrow aA$

$A \rightarrow b$

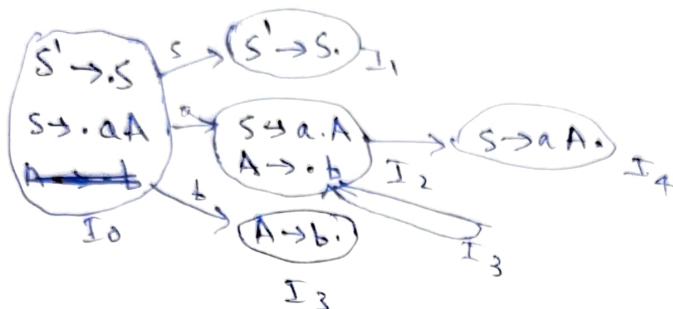
S1 : Augment grammar

$S' \rightarrow S$

$S \rightarrow aA$

$A \rightarrow b$

S-S :



S-6 :

$\text{first}(S) = \{a\}$

~~$\text{first}(S) = \{a\}$~~

$\text{first}(aA) = \{b\}$

~~$\text{follow}(S') = \{\$ \}$~~

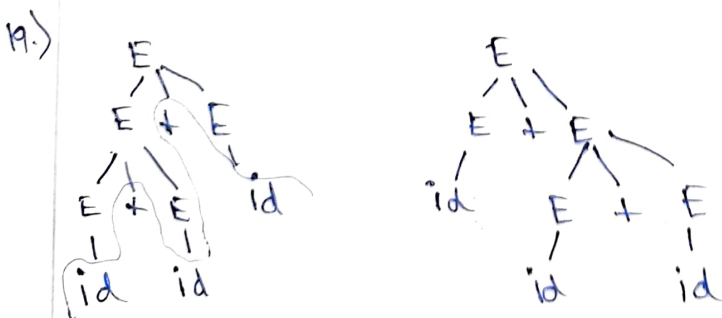
$\text{follow}(S) = \{\$ \}$

$\text{follow}(A) = \{\$ \}$

16) $\text{follow}(A) = \{b\}$

17) $S' \rightarrow S$
 $S \rightarrow a$

18) ~~$S' \rightarrow .S$~~
 $S \rightarrow .a$



Hence it is ambiguous

20) ~~S → A~~
~~S → B~~
~~A → A~~
~~B → B~~
 id + id
 Leftmost derivation:
 $E \rightarrow E + E$
 $E \rightarrow id + E$
 $E \rightarrow id + id$

21) Left Factoring:

$S \rightarrow a S'$
 $S' \rightarrow A | B$

22) $\text{First}(A) = \{x, y\}$

23) $S \rightarrow aA | aB$
 $A \rightarrow b$
 $B \rightarrow c$

Ans:

~~$\text{First}(aA) = \{a\}$~~ $\text{First}(aA) = \{a\}$
 ~~$\text{First}(aB) = \{a\}$~~ $\text{First}(aB) = \{a\}$

The First sets are identical

$S \rightarrow aS'$
 $S' \rightarrow A | B$
 $A \rightarrow b$
 $B \rightarrow c$

∴ The grammar is not suitable for Predictive Parsing.

24) $S() \{$
 if (input == 'a') {
 input++;
 $S()$
 }
 else if (input == 'b') {
 input++;
 }
 }
 main() {
 $S()$;
 if (input == 'S')
 printf("Parsing successful");
 }

$\$S$ $\$$

Accept

25) $S \rightarrow aS$
 $S \rightarrow b$

$\text{first}(S) = \{a, b\}$

$\text{follow}(S) = \{ \$ \}$

	a	b	\$
S	$S \rightarrow aS$	$S \rightarrow b$	-